

STM32 Hardware JPEG Peripheral

Questions & Answers — Complete Reference

19 technical questions with detailed answers

Based on STM32H5 JPEG_EncodingFromFLASH_DMA Analysis

Table of Questions

- Q1** Pourquoi le JPEG hardware nécessite-t-il une conversion RGB → YCbCr avant l'encodage ?
- Q2** Qu'est-ce que le sous-échantillonnage chroma 4:4:4, 4:2:2, et 4:2:0 ?
- Q3** Pourquoi MAX_INPUT_LINES vaut 16 pour le 4:2:0 et 8 pour les autres modes ?
- Q4** Quel est le rôle du check de taille d'image dans RGB_GetInfo() ?
- Q5** Quelle est la différence entre DataBufferSize et Jpeg_IN_BufferTab.DataBufferSize ?
- Q6** Pourquoi jpgBuffer est déclaré en uint32_t alors que RGB_IMAGE_SIZE est en octets ?
- Q7** Y a-t-il un bug dans la déclaration uint32_t jpgBuffer[RGB_IMAGE_SIZE] ?
- Q8** Quel est le rôle de CHUNK_SIZE_OUT (4096 octets) et comment le choisir ?
- Q9** Pourquoi le traitement se fait par chunks de 16 lignes et pas sur toute l'image ?
- Q10** Comment fonctionnent HAL_JPEG_GetDataCallback et HAL_JPEG_DataReadyCallback et quand se déclenchent-ils ?
- Q11** Pourquoi le DMA ne passe pas automatiquement aux 8 mots suivants sans HAL_JPEG_ConfigInputBuffer ?
- Q12** Peut-on utiliser DMA_TCEM_LAST_BLOCK_TRANSFER pour réduire le nombre d'IRQ ?
- Q13** Peut-on déplacer toute la logique des callbacks dans le main avec des flags ?
- Q14** Pourquoi les IRQ input et output sont-elles indépendantes — l'une ne peut-elle pas signaler l'autre ?
- Q15** Peut-on éliminer les chunks et convertir toute l'image en une seule fois ?
- Q16** Pourquoi le burst DMA est de 8 mots (32 octets) et pas plus ?
- Q17** Pourquoi MAX_INPUT_WIDTH est 800 alors que l'image fait seulement 320 pixels ?
- Q18** Pourquoi Image_RGB888 ne peut-il pas être déclaré en uint32_t comme les autres formats ?
- Q19** Quelles sont les familles STM32 avec un périphérique JPEG hardware ?

Q1 Pourquoi le JPEG hardware nécessite-t-il une conversion RGB → YCbCr avant l'encodage ?

Le JPEG hardware ne travaille pas en espace colorimétrique RGB. Il exige des données en YCbCr parce que la compression JPEG exploite une propriété de la vision humaine : l'œil est très sensible à la luminosité (Y) mais peu sensible aux variations de couleur (Cb, Cr). En séparant ces deux types d'information, JPEG peut réduire agressivement les données de couleur sans que l'œil humain ne perçoive de différence.

La conversion est effectuée par la bibliothèque jpeg_utils via un pointeur de fonction sélectionné automatiquement selon le format source (ARGB8888 / RGB888 / RGB565) et le mode de sous-échantillonnage choisi.

Q2 Qu'est-ce que le sous-échantillonnage chroma 4:4:4, 4:2:2, et 4:2:0 ?

La notation A:B:C décrit combien d'échantillons Cb et Cr sont conservés par rapport aux échantillons Y (luma). Dans tous les modes, chaque pixel garde son échantillon Y complet.

Mode	Cb/Cr par groupe	Taille MCU	MAX_INPUT_LINES	Taille fichier
4:4:4	1 échantillon / 1 pixel	8 × 8 px	8	100% (le plus grand)
4:2:2	1 échantillon / 2 pixels (horizontal)	16 × 8 px	8	~75%
4:2:0	1 échantillon / bloc 2×2 pixels	16 × 16 px	16	~50% (le plus petit)

Q3 Pourquoi MAX_INPUT_LINES vaut 16 pour le 4:2:0 et 8 pour les autres modes ?

En 4:2:0, un seul échantillon Cb/Cr couvre un bloc de 2×2 pixels — donc 2 lignes de hauteur. Le JPEG hardware ne peut pas encoder un MCU incomplet : il a besoin des 2 lignes simultanément pour accéder à l'échantillon chroma partagé. Le MCU devient donc 16×16 pixels, et MAX_INPUT_LINES doit être 16.

En 4:4:4 et 4:2:2, la hauteur du MCU est 8 pixels. 8 lignes suffisent pour former un MCU complet.

```
// Conséquence directe sur la taille du chunk d'entrée :
DataBufferSize = ImageWidth × MAX_INPUT_LINES × BYTES_PER_PIXEL
                = 320      × 16      × 4
                = 20 480 octets par chunk (ARGB8888, 4:2:0)
```

Q4 Quel est le rôle du check de taille d'image dans RGB_GetInfo() ?

La fonction vérifie 4 règles avant de démarrer l'encodage. Si une règle est violée, Error_Handler() est appelé immédiatement car le hardware JPEG ne peut pas encoder des blocs incomplets.

Règle	Condition	Raison
-------	-----------	--------

Largeur multiple de 8	Toujours	Le DCT travaille sur des blocs 8×8
Hauteur multiple de 8	Toujours	Même raison
Largeur multiple de 16	4:2:2 et 4:2:0	MCU de 16 pixels en largeur
Hauteur multiple de 16	4:2:0 uniquement	MCU de 16 pixels en hauteur

Q5 Quelle est la différence entre `DataBufferSize` et `Jpeg_IN_BufferTab.DataBufferSize` ?

DataBufferSize (variable locale) : Taille des données RGB brutes lues depuis la FLASH. Fixe et constante — $\text{ImageWidth} \times \text{MAX_INPUT_LINES} \times \text{BYTES_PER_PIXEL} = 20\,480$ octets. Sert à avancer le curseur `RGB_InputImageIndex` dans l'image source.

Jpeg_IN_BufferTab.DataBufferSize (champ struct) : Taille des données YCbCr produites après conversion et stockées dans le buffer d'entrée. Variable — écrite par `pRGBToYCbCr_Convert_Function`. C'est cette valeur que le DMA Channel 1 utilise comme taille de transfert.

Les deux diffèrent parce que la conversion RGB→YCbCr avec sous-échantillonnage 4:2:0 réduit la quantité de données chroma. 20 480 octets RGB en entrée ne produisent pas 20 480 octets YCbCr en sortie.

Q6 Pourquoi `jpgBuffer` est déclaré en `uint32_t` alors que `RGB_IMAGE_SIZE` est en octets ?

Raison 1 — Alignement DMA : Le DMA Channel 0 est configuré avec `DestDataWidth = DMA_DEST_DATAWIDTH_WORD` (32 bits). Il écrit en mémoire par mots de 4 octets. Un tableau `uint32_t` est automatiquement aligné sur 4 octets par le compilateur. Un tableau `uint8_t` pourrait être aligné sur 1 octet et provoquer un HardFault sur un accès DMA non aligné.

Raison 2 — Arithmétique pointeur : `pJpegBuffer` est `uint32_t*`. En C, `ptr+1` avance de `sizeof(*ptr) = 4` octets. Pour avancer de `DataBufferSize` octets réels, on divise par 4 : `pJpegBuffer += DataBufferSize / 4`.

Q7 Y a-t-il un bug dans la déclaration `uint32_t jpgBuffer[RGB_IMAGE_SIZE]` ?

Oui. `RGB_IMAGE_SIZE` est calculé en octets ($320 \times 240 \times 4 = 307\,200$ octets), mais il est utilisé comme nombre d'éléments `uint32_t`. Le compilateur alloue donc $307\,200 \times 4 = 1\,228\,800$ octets = 1,2 MB en SRAM — quatre fois la taille réelle nécessaire.

La même erreur se retrouve dans la déclaration du tableau `Image_ARGB8888` en FLASH, gaspillant 900 KB de mémoire Flash avec des zéros.

```
// Bug ST — alloue 1,2 MB :
uint32_t jpgBuffer[RGB_IMAGE_SIZE];           // RGB_IMAGE_SIZE = 307 200 octets

// Correct — alloue 300 KB :
uint32_t jpgBuffer[RGB_IMAGE_SIZE / 4];       // 76 800 éléments uint32_t
```

Sur STM32H5F5 (1,5 MB SRAM) le code passe par chance. Sur un MCU avec moins de SRAM ce bug provoquerait une erreur de linker ou un stack overflow au démarrage.

Q8 Quel est le rôle de `CHUNK_SIZE_OUT` (4096 octets) et comment le choisir ?

`CHUNK_SIZE_OUT` est la taille du buffer qui reçoit les données JPEG compressées en sortie du hardware. C'est une valeur arbitraire fixe — pas calculée. Elle représente la taille d'un chunk de sortie DMA.

4096 octets est un choix classique pour trois raisons : c'est une puissance de 2 (alignement DMA optimal), c'est dans les tailles de page standard (favorable au cache), et c'est suffisamment grand pour amortir les interruptions sans gaspiller de SRAM.

On peut utiliser 2048 (plus d'IRQ) ou 8192 (moins d'IRQ, plus de RAM). Ne pas descendre sous ~512 octets — le hardware JPEG produit des données par bursts de 32 octets minimum.

Q9 Pourquoi le traitement se fait par chunks de 16 lignes et pas sur toute l'image ?

C'est un compromis entre deux contraintes opposées :

- Contrainte minimum : le JPEG hardware ne peut pas encoder moins d'un MCU complet. En 4:2:0, un MCU fait 16 lignes. On ne peut pas descendre en dessous.
- Contrainte maximum : stocker toute l'image convertie en SRAM coûterait ~230 KB juste pour le buffer d'entrée, en plus du reste du programme.

Le système par chunks réutilise le même buffer de 51 200 octets à chaque itération. La FLASH contient l'image source complète, mais à aucun moment elle n'existe entièrement en SRAM sous forme YCbCr.

Q10 Comment fonctionnent `HAL_JPEG_GetDataCallback` et `HAL_JPEG_DataReadyCallback` et quand se déclenchent-ils ?

GetDataCallback : Déclenché par l'IRQ du DMA Channel 1 après chaque burst de 32 octets (8 mots). Appelé ~640 fois par chunk, ~9 600 fois pour une image 320×240 en 4:2:0. Si `NbEncodedData == DataBufferSize` → tout le chunk est consommé → EMPTY + Pause. Sinon → reconfigurer le DMA sur le reste du buffer.

DataReadyCallback : Déclenché quand le buffer de sortie est plein — le DMA Channel 0 a écrit 4096 octets de données JPEG compressées. Marque le buffer FULL, pause le DMA output, reconfigure le buffer pour le prochain chunk.

Les deux callbacks s'exécutent en contexte IRQ. Ils ne font que changer des flags et des états — aucun traitement lourd. Toute la logique réelle (conversion, memcpy) est dans les handlers CPU de la boucle principale.

Q11 Pourquoi le DMA ne passe pas automatiquement aux 8 mots suivants sans `HAL_JPEG_ConfigInputBuffer` ?

Avec `DMA_TCEM_BLOCK_TRANSFER`, le DMA entre en état SUSPEND après chaque burst et attend d'être réarmé explicitement. Il ne repart pas seul même s'il connaît l'adresse suivante.

`HAL_JPEG_ConfigInputBuffer` a donc deux rôles simultanés dans le else :

- Repositionner SAR (Source Address Register) sur la bonne adresse — car le GPDMA1 en mode SUSPEND ne conserve pas l'état mis à jour entre bursts.
-

- Sortir le canal de l'état SUSPEND via un bit RESUME dans le registre de contrôle — sans cela le DMA reste gelé indéfiniment même si JPEG_RX_REQ arrive.

Sans les deux paramètres (adresse + taille), le DMA repartirait depuis le début du buffer au lieu de continuer où il s'est arrêté.

Q12 Peut-on utiliser DMA_TCEM_LAST_BLOCK_TRANSFER pour réduire le nombre d'IRQ ?

Oui, et c'est une optimisation valide. DMA_BREQ_SINGLE_BURST gère seul la synchronisation avec le JPEG hardware en attendant JPEG_RX_REQ avant chaque burst — indépendamment du TransferEventMode.

	BLOCK_TRANSFER (ST)	LAST_BLOCK_TRANSFER (optimisé)
IRQ par chunk	640	1
IRQ image entière (320×240)	9 600	15
else branch	Utilisé 639×/chunk	Dead code — jamais atteint
ConfigInputBuffer	639×/chunk	0×/chunk
Risque FIFO overflow	Aucun	Aucun (BREQ_SINGLE_BURST)
Résultat JPEG	Identique	Identique

ST a choisi BLOCK_TRANSFER pour la portabilité entre familles et la détection d'anomalie en cours de chunk — pas pour une raison technique obligatoire.

Q13 Peut-on déplacer toute la logique des callbacks dans le main avec des flags ?

Oui, c'est une architecture valide et plus lisible. Les callbacks settent uniquement des flags volatile, et la boucle main lit ces flags pour décider des actions.

```
// Callbacks — zéro logique
void HAL_JPEG_GetDataCallback(JPEG_HandleTypeDef *hjpeg, uint32_t NbEncodedData)
{
    Flag_NbEncodedData = NbEncodedData;
    Flag_InputConsumed = 1;
}
void HAL_JPEG_DataReadyCallback(JPEG_HandleTypeDef *hjpeg,
                                uint8_t *pDataOut, uint32_t OutDataLength) {
    Flag_OutDataLength = OutDataLength;
    Flag_OutputReady    = 1;
}
```

Les flags DOIVENT être déclarés volatile sinon le compilateur peut les mettre en cache registre CPU et ne jamais voir la mise à jour faite par l'IRQ.

Q14 Pourquoi les IRQ input et output sont-elles indépendantes — l'une ne peut-elle pas signaler l'autre ?

Non. Les deux canaux DMA travaillent sur des FIFOs physiquement différents à des vitesses complètement différentes. La relation entre input consommé et output produit n'est pas 1:1 — elle dépend du contenu de l'image et du taux de compression.

	GetDataCallback	DataReadyCallback
Surveille	FIFO entrée consommé	FIFO sortie plein
Fréquence (320×240 4:2:0)	~9 600 fois	~75 fois
Répond à	"Le HW a lu mes données ?"	"Le HW a produit des données ?"

9 600 événements input pour seulement ~75 événements output. Ils ne peuvent clairement pas se remplacer.

Q15 Peut-on éliminer les chunks et convertir toute l'image en une seule fois ?

Techniquement oui, mais avec des compromis importants :

- InputHandler et GetDataCallback sont supprimés — plus de gestion de chunks.
- Un buffer de ~230 KB en SRAM est nécessaire pour stocker toute l'image YCbCr convertie.
- La conversion bloque le CPU pendant ~30ms avant que l'encodage puisse commencer — perte du parallélisme CPU/JPEG.
- Sur STM32F769 (512 KB SRAM) c'est impossible. Sur STM32H5F5 (1,5 MB) ça tient.

Le système par chunks existe précisément pour permettre l'encodage d'images sur des MCUs avec peu de SRAM tout en maximisant le parallélisme entre CPU et hardware JPEG.

Q16 Pourquoi le burst DMA est de 8 mots (32 octets) et pas plus ?

C'est un équilibre entre trois contraintes hardware :

- FIFO JPEG = 32 mots (128 octets). Avec burst = 8 mots, 4 bursts remplissent le FIFO. Si burst = 32 mots, un seul burst le remplirait — pas assez de marge pour que le JPEG hardware consomme avant le suivant.
- Limite GPDMA1 = 16 mots max par burst. 8 mots est en dessous de la limite et laisse de la marge.
- Arbitrage bus AHB : un burst de 8 mots monopolise le bus 8 cycles. Avec 16 ou 32 mots le CPU est bloqué 2 à 4 fois plus longtemps.

```
// Calcul de charge IRQ avec burst = 8 mots :
1 chunk = 20 480 octets / 32 octets par burst = 640 bursts
Image entière = 640 bursts × 15 chunks = 9 600 IRQ GetDataCallback
```

Q17 Pourquoi MAX_INPUT_WIDTH est 800 alors que l'image fait seulement 320 pixels ?

MAX_INPUT_WIDTH dimensionne le buffer IN alloué statiquement au compile time — pas la largeur de l'image encodée.

```
#define CHUNK_SIZE_IN (MAX_INPUT_WIDTH × BYTES_PER_PIXEL × MAX_INPUT_LINES)
                    = 800 × 4 × 16 = 51 200 octets
alloués

// Taille réellement utilisée à l'exécution :
DataBufferSize = Conf.ImageWidth × MAX_INPUT_LINES × BYTES_PER_PIXEL
               = 320 × 16 × 4 = 20 480 octets utilisés
```

800 permet d'encoder n'importe quelle image jusqu'à 800 pixels de large sans recompilation. C'est la résolution SVGA (800×600) — cible courante sur les boards STM32 avec afficheur.

Avec **MAX_INPUT_WIDTH** = 320, encoder une image 640×480 provoquerait un buffer overflow car **DataBufferSize** dépasserait la taille allouée.

Q18 Pourquoi Image_RGB888 ne peut-il pas être déclaré en uint32_t comme les autres formats ?

RGB888 utilise 3 octets par pixel — 3 n'est pas divisible par 4. Un tableau `uint32_t` ne peut pas représenter exactement des pixels de 3 octets sans perte ou padding.

La correction correcte pour RGB888 est d'utiliser `uint8_t` :

```
// ARGB8888 et RGB565 — divisibles par 4, uint32_t correct
const uint32_t Image_ARGB8888[RGB_IMAGE_SIZE / 4]; // exact
const uint32_t Image_RGB565 [RGB_IMAGE_SIZE / 4]; // exact (2 pixels par mot)

// RGB888 — 3 octets par pixel, uint8_t obligatoire
const uint8_t Image_RGB888 [RGB_IMAGE_SIZE]; // exact (pas de /4)
```

Q19 Quelles sont les familles STM32 avec un périphérique JPEG hardware ?

Seulement trois familles possèdent le codec JPEG hardware :

Famille	Lignes AVEC JPEG	Lignes SANS JPEG	DMA pour JPEG
STM32F7	F76x, F77x uniquement	Toutes les autres lignes F7	DMA2 Streams
STM32H7	H743/753, H745/755, H747/757, H750, H7A3/B3/B0, H7Rx/Sx	H723, H725, H730, H733, H735	MDMA / DMA2
STM32H5	H5E5, H5F5 uniquement	H503, H523, H533, H562, H563, H573	GPDMA1

Sur STM32H7 et STM32H5 (E5/F5), la DMA2D Chrom-Art peut convertir YCbCr→RGB en hardware après décodage. Sur STM32F7, cette conversion doit être faite en software.